

Exercício — Momento 3 (Prática aplicada, 120 min)

Aula 5 — Consumo de Web Services de Terceiros

Trabalhem na mesma squad, no mesmo repositório do projeto da clínica. Este momento tem duas metades bem diferentes: primeiro vocês DECIDEM e registram por escrito de quais serviços externos o desafio vai depender; depois vocês PROVAM que a decisão se sustenta, colocando uma dessas integrações para funcionar de verdade.

A primeira metade não é aquecimento para a segunda. O documento de decisão é entregável avaliado — e é onde a Capacidade Socioemocional desta aula é observada.

Passo 1 — Mapear as dependências externas (25 min)

Em squad, sem escrever código ainda. Peguem a lista de entregáveis do desafio e separem o que a clínica consegue resolver sozinha do que depende de terceiros.

1. Listem todas as funcionalidades do desafio que NÃO podem ser atendidas pela API própria da clínica. Comecem por mapa, push e SMS, e sigam adiante — CEP, geocoding e o que mais aparecer.
2. Para cada uma, respondam por escrito: por que a API da clínica não resolve isso?
3. Para cada uma, levantem pelo menos duas opções de fornecedor. Decisão com uma opção só não é decisão.
4. Para cada opção, procurem na documentação oficial: exige chave? qual é a cota gratuita? quais são os limites por segundo? o que os termos de uso exigem?
5. Anotem também o que acontece com o app se aquele serviço sair do ar amanhã. Essa resposta costuma ser a que muda a escolha.

Atenção a três armadilhas que aparecem todo semestre — vale conferir agora, e não na véspera da Aula 9:

- Push (Aula 9): o Expo Push Service não funciona no Expo Go no Android desde o SDK 53. Exige development build. Isso muda o planejamento da squad, não só o código.
- Mapas (Aula 12): o react-native-maps com Google Maps no Android exige chave de API, e a chave exige uma conta Google Cloud com faturamento habilitado — mesmo dentro da cota gratuita. Descobrir isso hoje é diferente de descobrir em 22/10.
- SMS (Aula 13): expo-sms abre o app de mensagens do aparelho e o usuário toca em enviar; um gateway (Twilio, Zenvia) envia pelo servidor, sem interação. São soluções diferentes para problemas diferentes. Decidam qual atende 'confirmação de consulta'.

Passo 2 — Registrar a decisão arquitetural (20 min)

Preencham o A05_05_Template_Decisao_Arquitetural.docx — um registro por serviço escolhido. Salvem no repositório da squad, em docs/decisoões/, e versionem junto com o código.

6. Contexto: que problema do desafio esse serviço resolve.
7. Opções consideradas: no mínimo duas, com o que cada uma cobra, limita e exige.
8. Decisão: qual foi escolhida, e o critério que pesou mais.
9. Consequências: o que fica mais fácil, o que fica mais arriscado, e o que acontece se o serviço sair do ar. Esta seção é obrigatória — é ela que separa um registro útil de uma anotação.
10. Plano B: o que o app faz enquanto o serviço estiver indisponível.

Escrevam para o colega que entrar na squad daqui a dois meses. Se o texto só faz sentido para quem estava na sala hoje, ele não serve.

Passo 3 — Cliente HTTP para serviços externos (15 min)

Objetivo: criar src/services/httpExterno.js. Ele é primo do api.js da Aula 4, mas não é o mesmo arquivo — e a diferença importa.

11. Criem a função buscarExterno(nomeServico, url, opcoes): ela aplica timeout com AbortController e envia o header User-Agent.
12. NÃO envie o token da clínica aqui. Mandar a credencial da nossa API para um servidor de terceiro é vazar credencial — é exatamente por isso que este arquivo é separado do api.js.
13. Tratem 429 (limite atingido) com mensagem própria: insistir depois de um 429 costuma render bloqueio maior.
14. Criem um erro próprio, ServicoExternoIndisponivel, guardando o nome do serviço. A tela precisa poder dizer QUAL serviço falhou.
15. Convertam o AbortError em uma mensagem de 'tempo esgotado' — 'AbortError' não significa nada para o usuário.

Referência: gabarito_httpExterno.js (em A05_07_Gabaritos.zip).

Passo 4 — Uma integração real funcionando (40 min)

Escolham UMA das duas trilhas abaixo e façam funcionar de verdade no app da clínica. Squad que terminar antes faz a outra.

Trilha A — CEP no cadastro do paciente

16. Criem src/services/viacep.js com consultarCep(cep), usando o buscarExterno do Passo 3.
17. Tratem os três casos conferidos no Momento 2: sucesso, 200 com erro:"true", e resposta que não é JSON.

18. Traduzam a resposta para o vocabulário de vocês (localidade vira cidade, e assim por diante) antes de devolver.
19. No formulário de paciente, disparem a consulta no `onBlur` do campo CEP, quando ele tiver 8 dígitos. Não dispare a cada tecla: isso queima cota e atrapalha quem digita.
20. Preencham logradouro, bairro, cidade e UF automaticamente — mas deixem os campos editáveis. Se o serviço falhar, o cadastro continua possível à mão.
21. Mostrem um indicador enquanto a consulta corre, e um aviso (não um erro fatal) quando ela falhar.

Referências: `gabarito_viacep.js` e `gabarito_PacienteForm_cep.js`.

Trilha B — Geocoding do endereço da clínica

22. Criem `src/services/geocoding.js` com `geocodificar({ logradouro, cidade, uf })`, usando o `buscarExterno` do Passo 3.
23. Enviem o User-Agent exigido pela política do Nominatim e tratem 403, 429 e lista vazia.
24. Guardem o resultado: o endereço da clínica não muda. Geocodificar uma vez e reaproveitar economiza cota e deixa a tela instantânea nas próximas aberturas.
25. Exibam as coordenadas em uma tela de 'Sobre a clínica'. Na Aula 12 elas viram um mapa de verdade — o trabalho de hoje não é jogado fora.
26. Incluam na tela o crédito ao OpenStreetMap. A política exige atribuição visível, e cumprir isso faz parte do exercício.
27. Se forem geocodificar vários endereços, espacem as chamadas em pelo menos 1 segundo. Nada de `Promise.all`.

Referência: `gabarito_geocoding.js`.

Passo 5 — Commit e Pull Request (10 min)

Cada integrante envia o seu trabalho na branch do próprio módulo, no padrão combinado na Aula 2. Os documentos de decisão vão versionados junto com o código:

```
git add .
git commit -m "feat: integra servico externo de CEP e registra decisao
arquitetural"
git push
```

Confirmam antes de commitar: nenhuma chave de API dentro do código. Se a squad já criou alguma chave para as próximas aulas, ela vai para um arquivo `.env` fora do Git, com um `.env.example` versionado sem os valores.

Passo 6 — Preparar o terreno das próximas aulas (se sobrar tempo)

Estas três providências levam dias, não minutos, e travam a aula quando ficam para a véspera. Quem adiantar agora agradece depois:

- Criar a conta no serviço de mapas e gerar a chave de API (Aula 12).

- Verificar se a squad tem pelo menos um aparelho Android capaz de rodar um development build, necessário para push (Aula 9) e para Bluetooth (Aula 7).
- Escolher o gateway de SMS e conferir se o plano gratuito atende ao volume de teste (Aula 13).

Ao terminar

Chamem o professor para validar rapidamente antes do Momento 4 (Fechamento). Tenham o documento de decisão aberto: a apresentação da squad é sobre ele.